tôi là newbie học java từ con số 0, hãy cung cấp cho tôi chi tiết kiến thức về "Method" trong java, để tôi biết và hiểu nó là gì? có tác dụng gì

Rất tốt khi bạn bắt đầu học Java từ con số 0 — “Method” (phương thức) là một trong những khái niệm **cốt lõi nhất** mà bạn cần hiểu rõ ngay từ đầu. Mình sẽ giải thích thật chi tiết, dễ hiểu, và có ví dụ nhé.

---

1. Method trong Java là gì?

Method (phương thức) là một **đoạn code được đặt tên**, dùng để thực hiện một nhiệm vụ cụ thể.

👉 Bạn có thể hiểu đơn giản:

- Method giống như một **công thức nấu ăn**
- Khi cần, bạn chỉ cần **gọi tên** là nó chạy

📌 Ví dụ đời thường:

- `nauCom()`
- `tinhTong()`
- `inXinChao()`

---

2. Tác dụng của Method

Method rất quan trọng vì nó giúp:

✅ 1. Tái sử dụng code

Viết một lần, dùng nhiều lần

```
void sayHello() {
    System.out.println("Xin chào!");
```

Java

Gọi nhiều lần:

```java
sayHello();
```

Java

✅ 2. Làm code gọn gàng, dễ đọc

Thay vì viết dài dòng, bạn chia nhỏ thành từng method

✅ 3. Dễ bảo trì

Sửa một chỗ → ảnh hưởng toàn bộ chương trình

---

3. Cấu trúc của một Method

Cú pháp chung:

```java
returnType tenMethod(thamSo) {
    // code
```

Java

Ví dụ:

```java
int tinhTong(int a, int b) {
    return a + b;
```

Java

👉 Giải thích:

- `int` → kiểu dữ liệu trả về
- `tinhTong` → tên method

- `(int a, int b)` → tham số
- `return` → trả kết quả

---

4. Các thành phần quan trọng

4.1. Kiểu trả về (Return type)

Kiểu	Ý nghĩa
`void`	Không trả về gì
`int`	Trả về số nguyên
`double`	Trả về số thực
`String`	Trả về chuỗi

4.2. Tên method

📌 Quy tắc:

- Viết theo **camelCase**
- Có ý nghĩa

Ví dụ:

```
tinhDiemTrungBinh()
```

Java

4.3. Tham số (Parameters)

Là dữ liệu truyền vào method

```
void sayHello(String name) {
    System.out.println("Xin chào " + name);
```

Java

Gọi:

Java

4.4. Return (giá trị trả về)

```
int square(int x) {
    return x * x;
```

Java

---

5. Phân loại Method

5.1. Method không có tham số

```
void hello() {
    System.out.println("Hi!");
```

Java

5.2. Method có tham số

```
void greet(String name) {
    System.out.println("Hello " + name);
```

Java

5.3. Method có return

```java
int sum(int a, int b) {
    return a + b;
```

Java

5.4. Method không return (void)

```java
void printSum(int a, int b) {
    System.out.println(a + b);
```

Java

---

6. Cách gọi Method

```java
public class Main {
    public static void main(String[] args) {
        sayHello();
    }

    static void sayHello() {
        System.out.println("Xin chào!");
    }
}
```

📌 Lưu ý:

- Trong `main`, thường gọi method bằng `static`

---

7. Method static và non-static

7.1. Static method

```
static void hello() {
```

Java

👉 Gọi trực tiếp:

Java

7.2. Non-static method

```
void hello() {
    System.out.println("Hello");
```

Java

👉 Phải tạo object:

```
Main obj = new Main();
```

Java

---

8. Ví dụ tổng hợp

```
public class Main {

    public static void main(String[] args) {
        int result = tinhTong(5, 3);
        System.out.println(result);
    }

    static int tinhTong(int a, int b) {
        return a + b;
    }
}
```

9. Một số lưu ý quan trọng

⚠️ Method phải nằm trong class
⚠️ Không thể viết method bên ngoài class
⚠️ `return` phải đúng kiểu
⚠️ Method có thể gọi method khác

---

10. Tóm tắt dễ nhớ

👉 Method = **hàm = chức năng nhỏ trong chương trình**

- Có tên
- Có thể có input (tham số)
- Có thể có output (return)
- Giúp code gọn, dễ dùng lại

Nếu bạn muốn học sâu hơn

Mình có thể dạy tiếp cho bạn các phần nâng cao hơn như:

- Overloading (nạp chồng method)
- Recursion (đệ quy)
- Pass by value
- Method trong OOP

👉 Bạn muốn mình giải thích tiếp phần nào không?

tham số và đối số là gì? hãy cung cấp kiến thức này cho mình

Đây là một câu hỏi **rất quan trọng** khi học Java — nhiều người mới thường nhầm giữa **tham số (parameter)** và **đối số (argument)**. Mình sẽ giải thích thật rõ, dễ hiểu kèm ví dụ 👇

1. Tham số (Parameter) là gì?

👉 **Tham số** là **biến được khai báo trong method**.

Nó đóng vai trò **nhận dữ liệu đầu vào** khi method được gọi.

📌 Ví dụ:

```java
void sayHello(String name) {
    System.out.println("Xin chào " + name);
```

Java

👉 Ở đây:

- `name` chính là **tham số**

---

2. Đối số (Argument) là gì?

👉 **Đối số** là **giá trị thực tế được truyền vào method khi gọi**.

📌 Ví dụ:

Java

👉 Ở đây:

- `"An"` là **đối số**

---

3. So sánh dễ hiểu

Tham số (Parameter)	Đối số (Argument)
Khai báo trong method	Truyền khi gọi method
Là biến	Là giá trị cụ thể

Tham số (Parameter)	Đối số (Argument)
Ví dụ: `String name`	Ví dụ: `"An"`

---

4. Ví dụ đầy đủ

```
public class Main {
    public static void main(String[] args) {
        greet("Nam");
    }

    static void greet(String name) {  // ← tham số
        System.out.println("Hello " + name);
    }
}
```

👉 Phân tích:

- `String name` → tham số
- `"Nam"` → đối số

---

5. Ví dụ với nhiều tham số

```
int sum(int a, int b) {
    return a + b;
```

Java

Gọi:

Java

👉 Ở đây:

- `a`, `b` → tham số

- `5, 3` → đối số

---

6. Cách nhớ cực dễ (mẹo)

👉 Bạn nhớ câu này:

- **Parameter = placeholder (chỗ trống)**
- **Argument = giá trị thật**

📌 Ví dụ:

Java

Java

---

7. Một số lưu ý quan trọng

⚠️ 1. Số lượng phải khớp

Java

⚠️ 2. Kiểu dữ liệu phải đúng

Java

⚠️ 3. Thứ tự phải đúng

Java

Java

---

8. Ví dụ thực tế hơn

```java
void register(String username, String password) {
    System.out.println("User: " + username);
```

Java

Gọi:

Java

👉

- `username` , `password` → tham số
- `"admin"` , `"123456"` → đối số

---

9. Tóm tắt siêu ngắn

👉 **Tham số (Parameter)**
= biến trong method

👉 **Đối số (Argument)**
= giá trị truyền vào method

Nếu bạn muốn, mình có thể giải thích thêm các phần nâng cao liên quan như:

- Truyền tham trị (pass by value) trong Java

- Truyền object vs primitive
- Lỗi thường gặp khi dùng parameter/argument

👉 Bạn muốn học tiếp phần nào?

cho một ví dụ đơn giản dễ hiểu

Ok, mình sẽ cho bạn một ví dụ **rất đơn giản, dễ hiểu nhất có thể** 👇

Ví dụ: In tên

```java
public class Main {
    public static void main(String[] args) {
        sayHello("An");  // ← đối số
    }

    static void sayHello(String name) {  // ← tham số
        System.out.println("Xin chào " + name);
    }
}
```

Giải thích cực dễ

👉 Khi bạn **khai báo method**:

Java

- `name` = **tham số** (biến dùng để nhận dữ liệu)

👉 Khi bạn **gọi method**:

Java

- `"An"` = **đối số** (giá trị thật truyền vào)

Hình dung đơn giản

- Method giống như **cái hộp**
- Tham số = **chỗ để bỏ đồ vào**
- Đối số = **đồ bạn bỏ vào**

👉 Ở đây:

- Hộp: `sayHello`
- Chỗ bỏ: `name`
- Đồ bỏ vào: `"An"`

Kết quả chạy

```
Xin chào An
```

Tóm lại 1 câu

👉 `name` (tham số) nhận giá trị `"An"` (đối số)

Nếu bạn muốn, mình có thể cho thêm:

- ví dụ với số (toán học)
- hoặc ví dụ đời thực dễ hình dung hơn

Bạn muốn kiểu nào?

cho một ví dụ khác xem nào

Ok, mình cho bạn một ví dụ **khác, rất dễ hiểu và gần gũi hơn** 👇

Ví dụ: Tính tổng 2 số

Java

```java
public class Main {
    public static void main(String[] args) {
        int result = tinhTong(3, 5);  // ← đối số
        System.out.println(result);
    }

    static int tinhTong(int a, int b) {  // ← tham số
        return a + b;
    }
}
```

Giải thích đơn giản

Khi khai báo method:

Java

```java
static int tinhTong(int a, int b)
```

- `a`, `b` → **tham số**
  👉 chỉ là "biến giữ chỗ", chưa có giá trị cụ thể

Khi gọi method:

Java

```
tinhTong(3, 5);
```

- `3, 5` → **đối số**
  👉 là giá trị thật được truyền vào

Quá trình chạy

- `a = 3`
- `b = 5`
- `return a + b` → `8`

👉 In ra:

```
8
```

Cách hiểu cực dễ

👉 Giống như phép toán:

- Công thức: `a + b` (tham số)
- Thay số: `3 + 5` (đối số)

Tóm lại

- `int a, int b` → tham số
- `3, 5` → đối số

Nếu bạn muốn, mình có thể cho thêm:

- ví dụ nhập từ bàn phím (thực tế hơn)
- hoặc ví dụ với String (chuỗi)
- hoặc bài tập nhỏ để bạn tự làm

👉 Bạn muốn thử làm 1 bài không?

ChatGPT
Đăng nhập
Đăng ký miễn phí